UNTRM

UNTRM · INCID · FIZBAC · 2026-II

ML e IA: de desarrollo a producción

Presentación y entorno · del dato al modelo, del modelo al servicio.

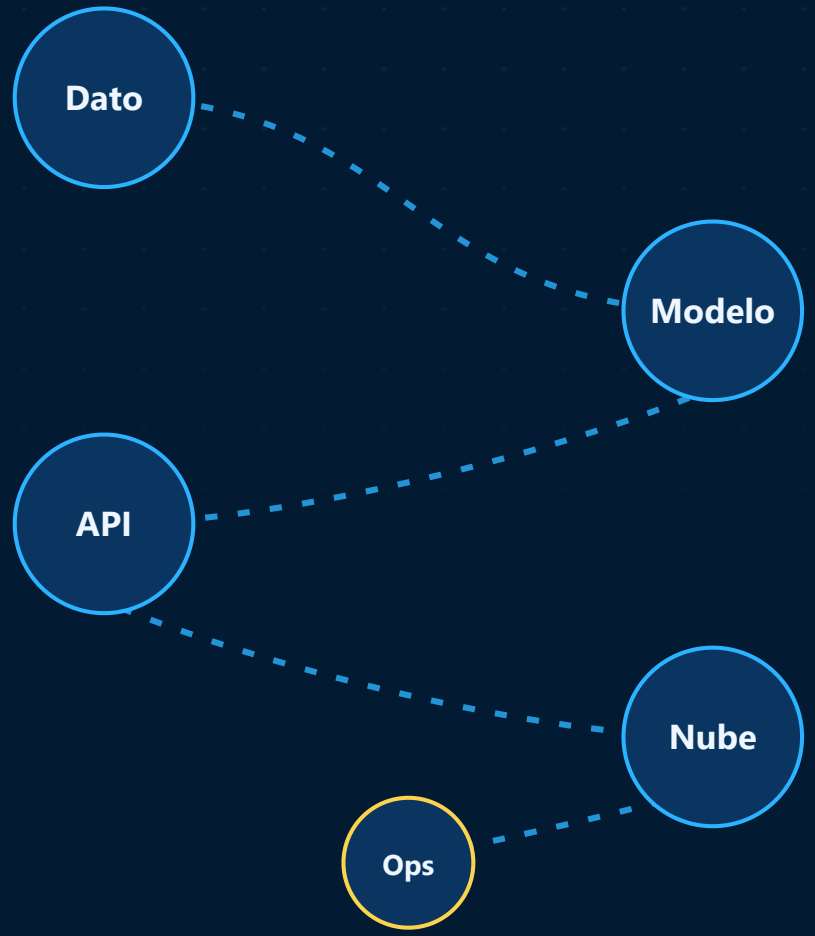


Helí Campos R.
BigData, Analytics & AI Architect
linkedin.com/in/helicr · helicr@gmail.com

NÚCLEO: ML CLÁSICO DE EXTREMO A EXTREMO

EN LÍNEA · GOOGLE MEET

22 HORAS SÍNCRONAS



APERTURA · 3 MIN

¿Desde dónde partimos?

Pregunta 1 de 3 · responde en voz alta o en el chat

¿Has llevado un modelo más allá del notebook, para que otra persona lo use?

A Todavía no: el trabajo termina en el cuaderno o en un script local.

B Lo intenté: hubo API, archivo o demo, pero no quedó reproducible.

C Sí: lo empaqué o lo serví, al menos en mi máquina.

D Sí: llegó a un entorno compartido o a la nube.

Del dato al servicio, y que otra persona lo pueda operar

NÚCLEO OBLIGATORIO ML clásico de extremo a extremo

- **Datos reproducibles** por capas: raw → master → fs → ml
- **Entrenamiento y seguimiento** con MLflow
- **API y contenedor** con FastAPI y Docker (en Codespaces)
- **De local a GCP:** migración y despliegue en Cloud Run
- **Pipelines con GitHub Actions** y CI/CD
- **MLOps:** versiones, monitoreo y rollback

OPCIONAL

- Red neuronal guiada (PyTorch)
- LLM abierto, como sesión extra
- RAG y la visión de LLMOps / AgentOps

Fuera del curso

MCP y agentes. Nada del curso exige una GPU propia.

A quién se dirige: investigadores, docentes y estudiantes que ya pueden entrenar un modelo y, con alta probabilidad, no lo han llevado a producción.

Del notebook al servicio

Historia hipotética

La predicción es correcta. El cuaderno vive en una ruta personal y las dependencias no están documentadas. Un colega quiere consumir el modelo; después cambian los datos o una versión.

¿Quién puede reproducirlo, usarlo y recuperarlo?

Avanza con → para revelar el camino. Los pasos 1–6 son el núcleo; el último es opcional.

1. **Datos y capas reproducibles** — raw → master → fs → ml: originales, validación y trazabilidad — S1
2. **Entrenamiento y artefactos** — Modelo, preproceso, métricas y metadatos; seguimiento con MLflow — S1
3. **API y contrato de inferencia** — El mismo preproceso; entradas validadas; errores explícitos — S2
4. **Contenedor y ejecución repetible** — Dependencias, configuración y pruebas fuera del notebook — S2
5. **Migración y despliegue en GCP** — Datos en Cloud Storage y BigQuery; servicio en Cloud Run — S3
6. **Orquestación, monitoreo, versiones y rollback** — GitHub Actions, registros, alertas, actualización y recuperación — S4–S5

\+ **La disciplina se extiende a LLMs y agentes** — Visión de LLMOps y AgentOps, fuera del núcleo — OPCIONAL

El modelo es la parte pequeña del sistema



45 % del tiempo se va en **cargar y limpiar datos**, antes de entrenar.

Sculley et al. (2015), *Hidden Technical Debt in ML Systems*. Anaconda (2020), 2 360 personas de más de 100 países: dato histórico de ciencia de datos.

MADUREZ DE MLOPS · GOOGLE CLOUD

0 **Nivel 0 · Proceso manual**
En el curso: S1–S2

1 **Nivel 1 · Automatizar el pipeline**
En el curso: S4

2 **Nivel 2 · CI/CD**
En el curso: S4–S5

Nivel 0 · Proceso manual S1–S2

El modelo se entrena y se entrega a mano, casi siempre desde un notebook. Scripts y pasos sueltos, sin pruebas ni registro.

Riesgo: el modelo se degrada sin que nadie lo note y es difícil reproducirlo.

Fuente: Google Cloud, *MLOps: continuous delivery and automation pipelines in machine learning*.

Seis sesiones, tres hilos



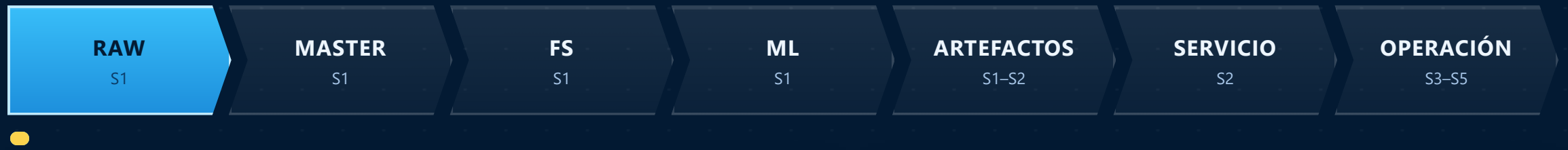
S1 De los datos al modelo 4 H

Comprender y ejecutar la preparación de datos y el entrenamiento.

- Capas raw, master, fs y ml
- Limpieza, variables y variable objetivo
- Particiones, entrenamiento y evaluación
- Seguimiento de experimentos con MLflow (versión ligera)

Evidencia: Avance en S2: capas, modelo, métricas y explicación.

Del archivo original al servicio en producción



RAW S1

El dato de origen copiado tal cual, sin tocar: un archivo o una base operacional.

`raw_* · data/raw`

- Se verifica su integridad con un checksum
- Nunca se edita: todo lo demás se regenera desde aquí

Cómo se recorre

Cada capa se construye a partir de la anterior y nunca modifica la original.

Dos casos: el del instructor (Online Retail II, en S1) y el tuyo (Bank Marketing, UCI), que desarrollas de S1 a S5.

Lo que hoy corre en local, mañana en GCP

S1–S2 · EN LOCAL	S3–S5 · EN GCP	ESTRATEGIA
Archivos originales data/raw	Cloud Storage bucket con versionado de objetos	REPLATFORM
SQLite · master_ fs_ ml_ capas transformadas	BigQuery o Cloud SQL si es transaccional	REPLATFORM
Imagen Docker + uvicorn servicio de la API	Artifact Registry + Cloud Run sin administrar servidores	REHOST
Archivo .env credenciales locales	Secret Manager + IAM cuentas de servicio con permisos mínimos	REPLATFORM
Scripts manuales o cron pipeline de datos y entrenamiento	GitHub Actions workflows orquestados, sin claves en el repo	REFACTOR
MLflow local SQLite + carpeta de artefactos	MLflow sobre GCP o Vertex AI como alternativa	REPLATFORM

Antes de migrar se evalúa; después de migrar se **valida** (conteos, esquemas y agregados) y se planifica el **rollback**.

Ver → ejecutar → adaptar → explicar

01

Ver

El instructor resuelve un caso en vivo, con decisiones en voz alta.

→

02

Ejecutar

Cada participante reproduce el procedimiento en su entorno, no solo observa.

→

03

Adaptar

Se aplica el mismo esquema a un caso semejante, acumulativo, como tarea.

→

04

Explicar

Al inicio de la siguiente sesión se presenta el avance: qué se hizo y por qué.

Ritmo de cada sesión S1–S5 (4 horas)



Trabajo autónomo orientativo: 2–3 horas entre sesiones. Quien se atrasa corta el hilo: cada entrega construye la próxima.

Un avance construye el siguiente

S2 — Pipeline de datos + modelo + métricas
Se revisa en S2 · tu caso: Bank Marketing

S3 — API + contenedor
Se revisa en S3

S4 — Migración a GCP y despliegue
Opcional: módulo neuronal

S5 — Pipeline orquestado con GitHub Actions
Se revisa en S5

+ — Repositorio final + informe
Tras S5

Una captura no basta

La evidencia es el rastro que permite a otra persona repetir el resultado y entender las decisiones.

CÓDIGO · ENTORNO · COMANDOS · PRUEBAS · README · DECISIONES JUSTIFICADAS

Una captura de pantalla, por sí sola, no demuestra que el entorno exista ni que el comando funcione.

Conocer, hacer y dejar un producto

20%

Conocimiento

Comprender arquitecturas, decisiones y límites. No se aprueba solo memorizando comandos.

40%

Desempeño

Resolver el caso: ejecutar, adaptar y explicar. El proceso cuenta tanto como el resultado.

40%

Producto

Un entregable reproducible: repositorio, pruebas, README e informe con decisiones justificadas.

El 80 % de la nota es **demostrar y documentar** lo que se hizo. El criterio de evidencia aplica a cada unidad. (Sujeto a validación institucional.)

S0 · PAUSA PROGRAMADA

Pausa para el café



TIEMPO RESTANTE

15:00

A · 25	Pausa · 15	B · entorno · 70	10

S0 · 120 min. Pausa de 15 minutos.

PARTE B · 70 MINUTOS · BLOQUE OBLIGATORIO

Hoy dejamos una máquina lista para el resto del curso

No adelantamos el entrenamiento. Comprobamos, paso a paso, que todo responde.



El ancho es proporcional a los minutos previstos.

Cada paso responde cuatro preguntas

¿Para qué sirve? · ¿Qué ejecuto? · ¿Qué debo ver? · ¿Qué reviso si falla?

Si algo falla, **no se improvisa en silencio**: se anota el comando, la salida y se comunica. Eso también es evidencia.

Tu equipo · Codespaces · Actions · GCP

TU EQUIPO

Datos y pruebas

Git y VS Code

Miniconda y Python

SQLite (visor)

gcloud, o Cloud Shell

CODESPACES

Docker y Kubernetes

docker, kind, kubectl

Imágenes y contenedores

Laboratorio de K8s

No se instala en el portátil

GITHUB ACTIONS

Pipelines

Pruebas y calidad

Pipeline raw → modelo

Construcción de la imagen

Despliegue con aprobación

GOOGLE CLOUD

Datos y servicio

Cloud Storage y BigQuery

Artifact Registry

Cloud Run

Secret Manager e IAM

Tu equipo

Es el lugar por defecto de S1 y de las pruebas unitarias: explorar, validar, entrenar el modelo clásico y comprobar la API antes de empaquetar.

`Git · VS Code · Miniconda · SQLite · gcloud (o Cloud Shell)`

Qué software, en qué sesión y para qué

Herramienta	S0	S1	S2	S3	S4	S5	Extra
Git y GitHub	●	●	●	●	●	●	
VS Code	●	●	●	●	●	●	
Miniconda y entorno	●	●	●	●	●	●	
SQLite y DB Browser	●	●		◐			
MLflow		●	●		●	●	
FastAPI y uvicorn			●	◐	◐		
Docker	✓		●	◐	◐		
kind y kubectl	✓			◌			
GitHub Actions			●		●	◐	
Google Cloud	✓			●	●	●	
PyTorch · LLM				◌			◌

● Se instala y se comprueba ✓ Solo se comprueba ● Se usa ◐ Apoya o se reutiliza
◌ Opcional o si hay tiempo

Git y GitHub OBLIGATORIO

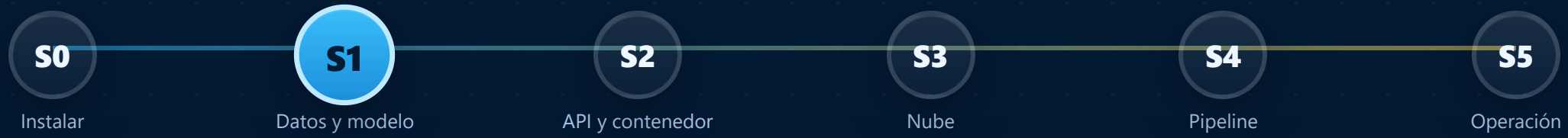
Guarda la historia de tu trabajo y te deja probar cambios en ramas sin romper lo que funciona. GitHub aloja tu repositorio y es donde corren los pipelines.

Dónde: Tu equipo y GitHub

S0	Cuenta de GitHub y clonar el repositorio
S1–S5	Cada entrega es una rama de tu repositorio (tarea-s1…)
S2 y S4	Un cambio subido puede lanzar los workflows de Actions

DISPONIBLE Guía de uso: **Git y ramas**

Qué estrena cada sesión y qué debe estar listo antes



S1 De los datos al modelo

SE ESTRENA

MLflow
comparar y registrar modelos

SIGUE EN USO

- Git y GitHub
- VS Code
- Miniconda y entorno
- SQLite y DB Browser

LISTO ANTES DE EMPEZAR

Diagnóstico de S0 completo: Git, VS Code, Miniconda y el entorno env-ia-mlops funcionando.

Todo corre en tu equipo, con el entorno del curso.

Herramientas que hay que tener listas

Todo | En tu equipo | En Codespaces | En la nube

VS Code
Editor del curso, con Python, Pylance, Jupyter, Container Tools, Kubernetes y Dev Containers.

OBLIGATORIO

Miniconda
Entornos aislados y reproducibles. No instalar paquetes sueltos en el base.

OBLIGATORIO

Git y GitHub
Historial, ramas y evidencias. GitHub Actions orquesta los pipelines.

OBLIGATORIO

DB Browser for SQLite
Inspeccionar raw, master, fs y ml sin escribir SQL a ciegas.

OBLIGATORIO

Docker (en Codespaces)
Construye y ejecuta la imagen dentro del Codespace. Docker Desktop local es opcional.

SIN INSTALAR

Google Cloud CLI
Autenticación y proyecto. En S0 solo se comprueba el acceso. Alternativa: Cloud Shell.

OBLIGATORIO

Instálalo **antes de S0** con la **guía de instalación para Windows** (`03-Entorno/guia-instalacion`). En clase solo verificamos: `instalar_entorno.cmd verificar` o `instalar_entorno.ps1 -Verificar`.

PARTE B · DESPUÉS DE INSTALAR

Instalar, aprender lo mínimo, comprobar

1 Instalar

Con la **guía de instalación para Windows**: `03-Entorno/guia-instalacion`.

2 Aprender lo mínimo

Con la **guía de uso** de cada herramienta: qué es, para qué sirve y la chuleta de comandos. En `07-Recursos/guias`.

3 Comprobar

Con el diagnóstico de esta Parte B: lo esperado frente a lo obtenido, y lo que falle, por escrito.

Miniconda y entornos

Crear, activar y reparar el entorno del curso.

DISPONIBLE

Git y ramas

Clonar, crear ramas, guardar y entregar cada tarea.

DISPONIBLE

Visual Studio Code

Extensiones, intérprete, kernel de notebooks y terminal.

DISPONIBLE

SQLite

Crear bases y tablas, lanzar SQL y verlas en una interfaz.

DISPONIBLE

Se publican antes de la sesión que las necesita

Antes de S1 · gcloud y Cloud Shell, Codespaces, MLflow, cuadernos y pruebas, terminal de Windows

Antes de S2 · Docker, FastAPI y GitHub Actions

Antes de S3 · Cloud Storage y BigQuery, Artifact Registry y Cloud Run, secretos e IAM, costes, kubectl y kind

S4 y S5 · Actions ampliada, autenticación sin claves, monitoreo, versionado y vuelta atrás

Clonar y abrir

Deja el material, los notebooks y los scripts en tu máquina, listos para abrirlos con el entorno del curso.

Terminal

```
$ git clone --branch develop
https://github.com/helicr/curso-ml-ia-
desarrollo-produccion.git
$ cd curso-ml-ia-desarrollo-produccion
$ code .
Cloning into 'curso-ml-ia-desarrollo-
produccion' ...
Receiving objects: 100%, done.
```

Demo ilustrativa de la salida esperada; la tuya puede variar.

Salida esperada

Una carpeta nueva y VS Code abriendo esa carpeta.

Si falla

Revisar HTTPS/SSH, la cuenta de GitHub y que no se haya recortado la URL. Si faltan carpetas, se clonó main: git fetch origin y git switch develop.

Miniconda

Se comprueba que Conda responde y se crea env-ia-mlops desde environment.yml (23 paquetes con versión fijada). Probado en Linux; en Windows, sin probar.

Terminal

```
$ conda --version
$ conda env list
$ conda env create -f 03-
Entorno\conda\environment.yml
conda <versión>
# conda environments:
base   *   /ruta/a/miniconda3
Executing transaction: done
```

Demo ilustrativa de la salida esperada; la tuya puede variar.

Salida esperada

Una versión de conda, la lista de entornos (al menos «base») y, al terminar la creación, «done». Después, env-ia-mlops aparece en conda env list.

Si falla

Conda no está en el PATH, la instalación no terminó o la terminal no recargó el perfil. Cerrar y abrir la terminal, o reiniciar VS Code, suele bastar.

Crea el entorno solo desde ese archivo y solo con conda-forge; no armes uno aparte con paquetes sueltos. Paso a paso, con las salidas reales: «Tu equipo listo para S1» (pasos 5 a 7).

VS Code y los notebooks

Hay que seleccionar el mismo entorno en un script y en un notebook. Si no, el cuaderno «funciona» con paquetes distintos a los del resto del curso.

Python

```
>>> import sys
>>> print(sys.executable)
C:\Users\tu-usuario\miniconda3\envs\env-ia-mlops\python.exe
```

Demo ilustrativa de la salida esperada; la tuya puede variar.

Salida esperada

Una ruta que apunte al entorno del curso (cuando exista), no al Python del sistema ni a uno improvisado.

Si falla

Command Palette → «Python: Select Interpreter» y «Select Notebook Kernel». Instalar las extensiones Python, Pylance y Jupyter si el selector no aparece.

SCRIPT DE LA BASE DE PRUEBA PENDIENTE

SQLite: comprobar ejecución, no calidad

Las capas transformadas vivirán en SQLite. Hoy solo se verifica que se puede abrir una base y ejecutar SQL.

SQLite

```
sqlite> SELECT 1 AS entorno_listo;
entorno_listo
1
```

Demo ilustrativa de la salida esperada; la tuya puede variar.

Salida esperada

Una fila con el valor 1 en la columna entorno_listo.

Si falla

El visor no está instalado, se abrió un archivo que no es una base, o el motor no está disponible en la terminal. Esta prueba no dice nada sobre la calidad de los datos del caso.

DENTRO DEL CODESPACE

Docker en Codespaces: una ejecución real

Docker no se instala en el portátil: se usa dentro del Codespace del curso. Hay que ver el motor en marcha y que pueda descargar y ejecutar una imagen. Docker Desktop local es opcional.

Terminal

```
$ docker version
$ docker run --rm hello-world
Client:  Version <x>
Server:  Engine <x>
Hello from Docker!
This message shows that your installation
appears to be working correctly.
```

Demo ilustrativa de la salida esperada; la tuya puede variar.

Salida esperada

Versiones de cliente y servidor, y el mensaje clásico de hello-world. La primera vez descarga la imagen.

Si falla

El Codespace no arranca, la cuota de GitHub se agotó o se abrió un entorno genérico, sin el .devcontainer del curso. Si usas Docker en local (opcional): Docker Desktop apagado, virtualización o WSL2 sin habilitar. Anota el error completo; no sustituyas la prueba por una captura del icono.

Google Cloud: autenticar, no gastar

Sustituye el marcador por el identificador real de tu proyecto. El objetivo de hoy es dejar la CLI reconocida, no crear servicios.

Terminal

```
$ gcloud auth login
$ gcloud config set project TU_ID_DE_PROYECTO
$ gcloud auth list
Credentialed Accounts
ACTIVE  ACCOUNT
*       tu-cuenta@ejemplo.com
```

Demo ilustrativa de la salida esperada; la tuya puede variar.

Salida esperada

Un flujo de autenticación en el navegador y, al final, tu cuenta marcada como activa en «gcloud auth list».

Si falla

CLI no instalada, cuenta distinta a la del proyecto, o identificador mal copiado. Alternativa: usar Cloud Shell desde el navegador.

No crear recursos de pago en esta comprobación. Cloud Storage, BigQuery, Artifact Registry y Cloud Run llegan en S3, con presupuesto y alertas.

Codespaces: Docker, kind y kubectl

Docker y Kubernetes no se instalan como requisito local. El repositorio trae un devcontainer (carpeta .devcontainer) con Docker, kind, kubectl y el entorno env-ia-mlops. En GitHub: Code → Codespaces → Create codespace on develop. Allí se hacen las prácticas de contenedores.

Terminal

```
$ docker version
$ kind version
$ kubectl version --client
$ bash .devcontainer/verificar.sh
Client:  Version <x>
kind v<x>
Client Version: v<x>
Fallos: 0
```

Demo ilustrativa de la salida esperada; la tuya puede variar.

Salida esperada

Tres versiones coherentes dentro del Codespace, no en el portátil personal, y «Fallos: 0» al final de verificar.sh. «kubectl version --client» no necesita un clúster encendido.

Si falla

Las cuotas de GitHub lo bloquean, la cuenta de empresa no permite Codespaces, o se abrió un entorno genérico en lugar del del curso. Comunicarlo: hay plan de respaldo (Cloud Shell o trabajar en pareja).

OPCIONAL

Red neuronal y LLM: ruta opcional

Sin GPU propia

Los módulos opcionales de redes (parte de S3) y de LLMs abiertos (sesión extra) se piensan para CPU o para un entorno compartido.

Modelo ligero o entorno compartido

Se usará un modelo que quepa en el equipo medio del grupo, o una ruta de respaldo publicada por el instructor.

Aún hay piezas por cerrar

Motor de inferencia y modelos definitivos están pendientes. Hoy basta con entender la ruta.

Este bloque es opcional: **no instales nada de esto hoy**. RAG es una ampliación posterior; MCP y agentes quedan fuera del curso. No adelantes descargas pesadas «por si acaso».

Resultado esperado y resultado obtenido

- [ ] **Git instalado y cuenta en GitHub**
  git --version · autenticación lista
- [ ] **VS Code y extensiones del curso**
  Python, Pylance, Jupyter, Container Tools, Kubernetes, Dev Containers
- [ ] **Miniconda responde**
  conda --version · conda env list
- [ ] **Repositorio clonado y abierto**
  git clone · code .
- [ ] **Entorno conda del curso creado**
  env-ia-mlops activo · prueba de humo: HUMO OK
- [ ] **Intérprete y kernel seleccionados**
  import sys; print(sys.executable)
- [ ] **SQLite comprobable**
  SELECT 1 AS entorno_listo; · DB Browser
- [ ] **Docker operativo en Codespaces**
  docker version · docker run --rm hello-world
- [ ] **gcloud autenticado y proyecto fijado**
  Sin crear recursos de pago
- [ ] **Codespaces del curso abierto**
  docker, kind, kubectl · bash .devcontainer/verificar.sh
- [ ] **Ruta neural / LLM comprendida** · opcional
  Sin GPU · no bloquea S1
- [ ] **Incidencias registradas y comunicadas**
  Esperado vs. obtenido



✓ correcto · ✗ fallo
Lista en el checklist.

Caso 1 · Online Retail II · el que desarrollamos

¿Qué vamos a hacer?

- Una tienda online quiere saber **a quién escribir**: ¿este cliente volverá a comprar en los próximos **90 días**?
- Yo lo resuelvo en vivo en **S1**, capa a capa: RAW → MASTER → FS → ML, y entrenamos el modelo.
- Tú lo ejecutas en tu equipo y haces la **tarea de S1**: añadir una variable y decidir con evidencia.

¿Qué hay en el archivo?

- Cada fila es una **línea de factura**: un producto dentro de una compra.
- 8 columnas: `Invoice`, `StockCode`, `Description`, `Quantity`, `InvoiceDate`, `Price`, `Customer ID`, `Country`.
- Ejemplo: factura 489547, 18 unidades de «PAPER CHAIN KIT 50'S CHRISTMAS» a 2,95, cliente 13599, Reino Unido.
- Trae suciedad real: cancelaciones, duplicados y compras sin cliente.

Dónde está: `04-Proyecto-Base/data/raw/muestra_online_retail_ii.csv` (muestra de 23 070 filas, ya en el repositorio). El archivo completo, 1 067 371 filas en Excel, se descarga con `python -m ia_mlops.dataset descargar`.

Caso 2 · Bank Marketing · el de tu tarea

¿Qué vas a hacer?

- Un banco portugués llamó por teléfono para vender un **depósito a plazo**. Pregunta: ¿este cliente lo **contratará**?
- Repites **la misma receta** que yo hago en S1, pero con tus datos y de S1 a S5.
- Lo presentas como **avance en S2** y después lo llevas a API, contenedor, GCP y workflow.

¿Qué hay en el archivo?

- Cada fila es una **llamada**: 41 188 filas, 20 columnas y la respuesta `y` (sí o no). Separador `;`.
- Ejemplo: cliente de 56 años (`job` housemaid, `marital` married), por teléfono, un lunes de mayo, 261 segundos → `y` = no.
- Solo un **11 %** dice «sí».
- Dos reglas: **no uses `duration`** (se sabe después de llamar) y **no reordenes las filas** (el orden es la fecha).

Dónde está: `02-Proyecto-Transversal/datos/bank-marketing/bank-additional-full.csv`; las columnas se explican en `bank-additional-names.txt`, en la misma carpeta. Opcional para investigar por tu cuenta: `datos/default-credit-card/`.

CIERRE · 10 MINUTOS

Antes de S1

1 · Completar el diagnóstico

Las doce comprobaciones (la de la ruta neural/LLM es opcional), con nota de lo que falló y lo que se obtuvo.

2 · Comunicar bloqueos

No esperar a S1 para decir que Codespaces, gcloud o la cuenta de la nube no arrancan.

3 · Leer el caso de Bank Marketing

Lee el enunciado y `bank-additional-names.txt`; comprueba la huella del archivo. No adelantes el entrenamiento ni «limpies» por intuición.

4 · No improvisar el entorno

Crea el entorno solo desde environment.yml; no armes uno paralelo con paquetes sueltos. El entorno del curso es env-ia-mlops.

S1 empieza con datos y entrenamiento. Quien llegue sin entorno perderá la práctica. En S1 trabajamos el caso del instructor (Online Retail II); tu caso es Bank Marketing y lo desarrollas de S1 a S5.

CIERRE · FIN DE LA SESIÓN PRINCIPAL

Nos vemos en S1, con el entorno listo

Tu tarea antes de S1

Completa la preparación, comprueba el entorno y lee el enunciado del caso (Bank Marketing, en `02-Proyecto-Transversal/`). Comunica los bloqueos con el comando y la salida obtenida.

Próximo paso: datos → modelo

En S1 construiremos las capas de datos, haremos el seguimiento con MLflow, entrenaremos y evaluaremos.

Anexos y material de consulta →

Plan docente

Repositorio del curso

Los anexos (estadísticas, costes, calculadora de Cloud Run y fuentes) son de consulta opcional y no forman parte de la sesión principal.